

FICHE DE RÉALISATION

Déploiement d'un cluster Kubernetes K3s complet

GitOps • DevSecOps • Monitoring • Stockage distribué

Étudiant :	Tom Marcorelli
Formation :	BTS SIO option SISR
École :	Nexa Digital School – Paris
Année :	2025 – 2026
Environnement :	VMware Workstation – 4 VMs Debian 12 – VMnet11 192.168.110.0/24

1. Présentation du projet	3
Contexte	3
2. Objectifs techniques	3
3. Schéma de l'infrastructure	4
4. Réalisation technique	4
Étape 1 – Préparation de l'infrastructure réseau	4
Étape 2 – Déploiement du cluster K3s	5
Étape 3 – Stockage persistant (Longhorn)	5
Étape 4 – Monitoring (Prometheus + Grafana + Alertmanager)	5
Étape 5 – Certificats TLS (Cert-Manager)	5
Étape 6 – Pipeline GitOps (Gitea + ArgoCD)	6
Étape 7 – Registry privée et scan CVE (Harbor + Trivy)	6
Étape 8 – Politiques de sécurité (Kyverno)	6
Étape 9 – Load Balancer bare-metal (MetalLB)	7
5. Tests et Validation	7
6. Bilan et Compétences	17

1. Présentation du projet

Contexte

Dans le cadre du BTS SIO option SISR, ce projet consiste à déployer une infrastructure Kubernetes complète en environnement virtualisé sur VMware Workstation. Il reproduit une architecture de production moderne basée sur K3s, la distribution Kubernetes légère certifiée CNCF, avec 4 machines virtuelles Debian 12 interconnectées sur un réseau dédié VMnet11.

Problématique

Comment déployer et opérer un cluster Kubernetes multi-nœuds en intégrant les pratiques DevOps (GitOps), DevSecOps (scan CVE, politiques d'admission) et Monitoring (métriques, alertes) sur une infrastructure bare-metal virtualisée ?

Stack technique déployée

Composant	URL / Accès	Rôle
K3s v1.35.5	kubectl get nodes	Orchestrateur Kubernetes – 1 master + 2 workers + 1 storage
Longhorn	https://longhorn.k3s.local	Stockage persistant distribué – StorageClass par défaut
Prometheus	https://prometheus.k3s.local	Collecte et stockage des métriques du cluster
Grafana	https://grafana.k3s.local	Dashboards de visualisation des métriques en temps réel
Alertmanager	https://alertmanager.k3s.local	Gestion et routage des alertes
Gitea	https://gitea.k3s.local	Forge Git auto-hébergée – dépôt k3s-apps
ArgoCD	https://argocd.k3s.local	Déploiement continu GitOps – synchronisation automatique
Harbor	https://harbor.k3s.local	Registry Docker privé + scan CVE Trivy
WordPress	https://wordpress.k3s.local	Application CMS de démonstration + MariaDB
Cert-Manager	–	Certificats TLS auto-signés pour tous les Ingress
Kyverno	–	Politiques de sécurité – Admission Controller K8s
MetalLB	192.168.110.100-120	Load Balancer bare-metal – IPs externes réelles
Traefik	–	Ingress Controller intégré K3s
Helm v3.21	–	Gestionnaire de paquets Kubernetes

2. Objectifs techniques

- Déployer un cluster K3s multi-nœuds (1 master + 2 workers + 1 storage) sur réseau VMnet11
- Mettre en place du stockage persistant distribué avec réplication multi-nœuds (Longhorn)

- Déployer un monitoring complet : collecte métriques (Prometheus), dashboards (Grafana), alertes (Alertmanager)
- Implémenter un pipeline GitOps de bout en bout : dépôt Git (Gitea) → déploiement automatique (ArgoCD)
- Mettre en place une registry Docker privée avec scan CVE automatique (Harbor + Trivy)
- Sécuriser le cluster avec des politiques d'admission K8s (Kyverno) en mode Enforce
- Gérer les certificats TLS de tous les services automatiquement (Cert-Manager)
- Exposer des services avec de vraies IPs externes sur un cluster bare-metal (MetalLB)

3. Schéma de l'infrastructure

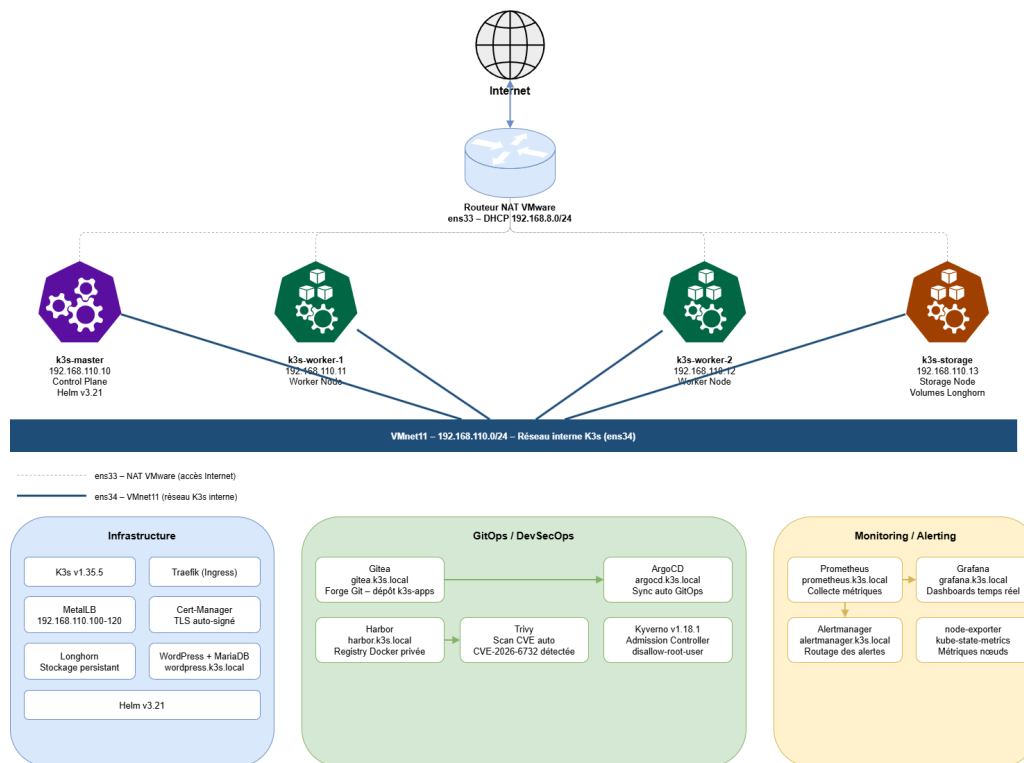


Schéma de l'infrastructure Lab K3s – Internet → Routeur NAT VMware → 4 VMs Debian 12 sur VMnet11 → Composants K3s

4. Réalisation technique

Étape 1 – Préparation de l'infrastructure réseau

VM	Rôle	CPU	RAM	Disque	IP VMnet11
k3s-master	Control Plane + Helm	2 vCPU	7 Go	40 Go	192.168.110.10
k3s-worker-1	Worker Node	2 vCPU	7 Go	40 Go	192.168.110.11
k3s-worker-2	Worker Node	2 vCPU	7 Go	40 Go	192.168.110.12
k3s-storage	Storage Node (Longhorn)	2 vCPU	7 Go	40 Go	192.168.110.13

■ Chaque VM a deux interfaces réseau : ens33 en DHCP sur le réseau NAT VMware pour accéder à Internet et télécharger les paquets, et ens34 en IP statique sur VMnet11 pour les communications internes K3s. Sans cette double interface, les nœuds ne pourraient pas rejoindre le cluster ET accéder aux dépôts apt.

```
# Vérification de la configuration réseau sur chaque VM
ip a

# Test de connectivité inter-nœuds (obligatoire avant K3s)
ping -c 3 192.168.110.11 # master → worker-1
ping -c 3 192.168.110.12 # master → worker-2
ping -c 3 192.168.110.13 # master → storage
```

Étape 2 – Déploiement du cluster K3s

■ K3s est choisi car il est packagé en un seul binaire < 100 Mo, idéal pour un lab à ressources limitées. Il intègre nativement Traefik comme Ingress Controller. Le script get.k3s.io automatise toute la configuration systemd.

```
# Sur le master : installation du control plane
curl -sfL https://get.k3s.io | sh -

# Récupération du token partagé pour les workers
sudo cat /var/lib/rancher/k3s/server/node-token

# Sur chaque worker et k3s-storage
curl -sfL https://get.k3s.io | K3S_URL=https://192.168.110.10:6443 K3S_TOKEN= sh -

# Vérification depuis le master
sudo kubectl get nodes
```

Étape 3 – Stockage persistant (Longhorn)

■ Par défaut, Kubernetes ne fournit pas de stockage persistant. Sans Longhorn, si un pod MariaDB redémarre, ses données sont perdues. Longhorn crée des volumes avec réplication sur plusieurs nœuds : même si un nœud tombe, les données restent accessibles. Helm gère automatiquement les 20+ ressources K8s nécessaires.

```
helm repo add longhorn https://charts.longhorn.io && helm repo update
helm install longhorn longhorn/longhorn --namespace longhorn-system --create-namespace
kubectl get pods -n longhorn-system
kubectl get storageclass
```

Étape 4 – Monitoring (Prometheus + Grafana + Alertmanager)

■ *Un cluster sans monitoring est aveugle. Le chart kube-prometheus-stack installe en une commande Prometheus (collecte), Grafana (visualisation), Alertmanager (alertes), node-exporter et kube-state-metrics.*

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
kubectl create namespace monitoring
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack --namespace monitoring
kubectl get svc -n monitoring
kubectl apply -f grafana-ingress.yaml
kubectl apply -f all-ingress-https.yaml
```

Étape 5 – Certificats TLS (Cert-Manager)

■ *Exposer des services en HTTP simple est une mauvaise pratique : les navigateurs bloquent les requêtes non sécurisées et ArgoCD refuse de fonctionner sans HTTPS. Cert-Manager surveille les Ingress annotés et génère les Secrets TLS sans intervention manuelle.*

```
sudo kubectl apply -f
https://github.com/cert-manager/cert-manager/releases/download/v1.14.0/cert-manager.yaml
kubectl apply -f cluster-issuer.yaml
kubectl get pods -n cert-manager
```

Étape 6 – Pipeline GitOps (Gitea + ArgoCD)

■ *Le GitOps est un paradigme où Git est la source de vérité unique. ArgoCD surveille le dépôt k3s-apps sur Gitea et synchronise automatiquement le cluster à chaque commit. L'état réel du cluster correspond toujours à ce qui est décrit dans Git.*

```
helm install gitea gitea-charts/gitea --namespace gitea --create-namespace
kubectl create namespace argocd
kubectl apply -n argocd -f
https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
kubectl apply -f argocd-app.yaml
kubectl get applications -n argocd
```

Étape 7 – Registry privée et scan CVE (Harbor + Trivy)

■ *Les images publiques Docker Hub peuvent contenir des CVE connues. Harbor est une registry privée qui intègre Trivy pour scanner automatiquement chaque image dès le push. Objectif : démontrer le cycle DevSecOps complet – détection CVE sur v1, correction Dockerfile, validation sur v2.*

```
helm install harbor harbor/harbor --namespace harbor --create-namespace \
--set expose.type=ingress --set expose.ingress.hosts.core=harbor.k3s.local \
--set externalURL=https://harbor.k3s.local \
--set persistence.persistentVolumeClaim.registry.storageClass=longhorn
# Build et push myapp:v1 (avec vulnérabilité CVE-2026-6732)
docker build -t harbor.k3s.local/k3s-lab/myapp:v1 .
docker push harbor.k3s.local/k3s-lab/myapp:v1
# Correction : RUN apk upgrade --no-cache dans le Dockerfile
docker build -t harbor.k3s.local/k3s-lab/myapp:v2 .
```

```
docker push harbor.k3s.local/k3s-lab/myapp:v2
```

Étape 8 – Politiques de sécurité (Kyverno)

■ Par défaut, n'importe quel utilisateur peut déployer un pod en root – faille critique. Kyverno s'installe comme Admission Webhook : il intercepte les requêtes avant acceptation par l'API Server et les rejette si elles violent une politique. Mode Enforce = blocage immédiat et automatique.

```
helm install kyverno kyverno/kyverno --namespace kyverno --create-namespace
kubectl apply -f kyverno-policy.yaml

# Test : tentative de créer un pod en root
sudo kubectl run test-root --image=nginx --namespace=default
# → Error from server: admission webhook denied the request
```

Étape 9 – Load Balancer bare-metal (MetalLB)

■ Sur un cluster bare-metal, les services LoadBalancer restent en état "Pending" faute d'un cloud provider. MetalLB résout ce problème en mode Layer 2 : il répond aux requêtes ARP pour la plage d'IPs réservée, rendant les services accessibles directement depuis Windows.

```
sudo kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.14.5/config/manifests/metallb-native.yaml
kubectl apply -f metallb-pool.yaml # IPAddressPool 192.168.110.100-120
kubectl apply -f metallb-l2.yaml # L2Advertisement
kubectl get svc nginx-lb # Vérification IP externe assignée
```

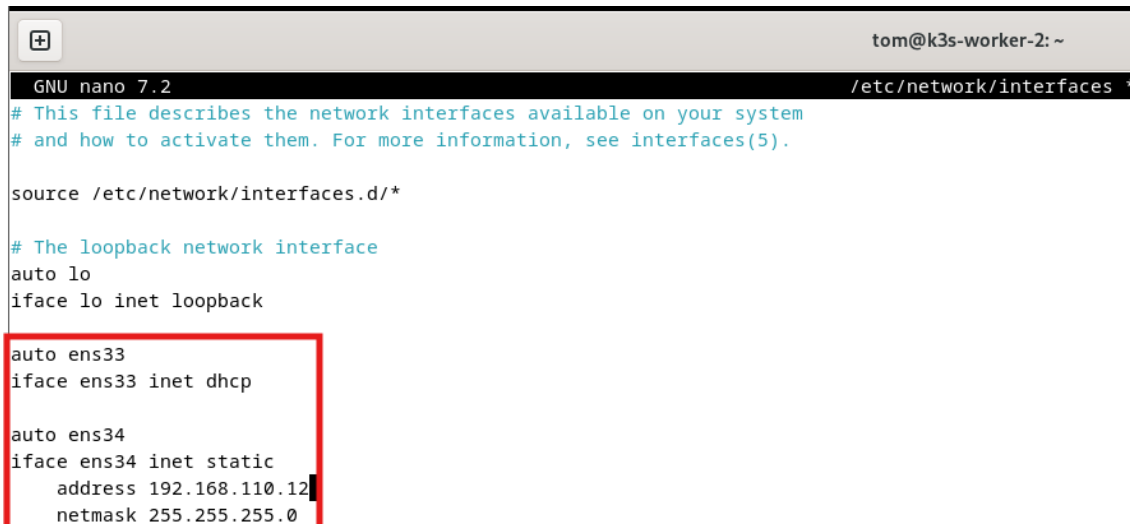
5. Tests et Validation

Test effectué	Résultat attendu	Validation
kubectl get nodes	4 nœuds Ready – v1.35.5+k3s1	■ Cluster K3s opérationnel
Ping inter-nœuds (VMnet11)	0 % packet loss, ~0,5 ms	■ Réseau interne fonctionnel
Dashboard Longhorn	2 volumes Healthy, 4 nœuds	■ Stockage persistant actif
https://grafana.k3s.local	Dashboards K8s temps réel	■ Monitoring opérationnel
Commit Gitea → ArgoCD	Sync automatique < 3 min	■ Pipeline GitOps fonctionnel
Harbor scan myapp:v1	CVE-2026-6732 Haute détectée	■ Scan Trivy opérationnel
Harbor scan myapp:v2	Aucune vulnérabilité	■ Correction CVE validée
kubectl run test-root	Pod bloqué par Kyverno	■ Politique sécurité active
Service LoadBalancer nginx	IP 192.168.110.101 assignée	■ MetalLB fonctionnel
https://wordpress.k3s.local	WordPress + MariaDB accessible	■ App + PVC Longhorn OK

Test effectué	Résultat attendu	Validation
Certificats TLS (tous services)	Cert valide généré automatiquement	■ Cert-Manager opérationnel

Preuve n°1 – Configuration réseau des VMs

■ On vérifie que chaque VM dispose bien de ses deux interfaces : *ens33* (NAT) et *ens34* (VMnet11 statique). C'est le prérequis absolu avant toute installation K3s.



```

tom@k3s-worker-2: ~
GNU nano 7.2 /etc/network/interfaces *
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

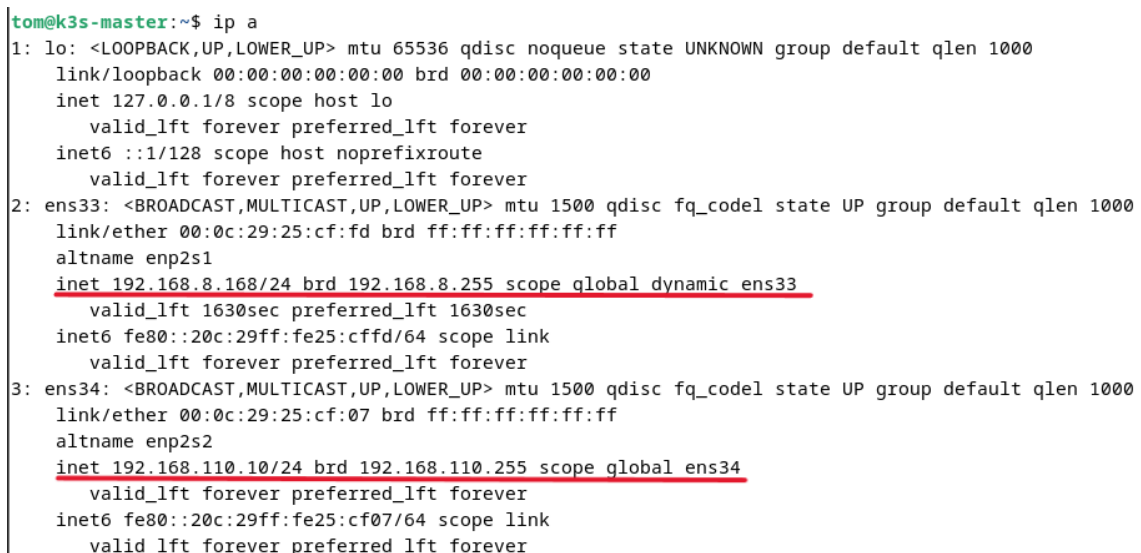
auto ens33
iface ens33 inet dhcp

auto ens34
iface ens34 inet static
    address 192.168.110.12
    netmask 255.255.255.0

```

Fig. 1 – /etc/network/interfaces sur k3s-worker-2 – ens33 DHCP + ens34 statique 192.168.110.12

■ On voit les deux blocs de configuration : *ens33* en DHCP pour le NAT VMware (accès Internet), et *ens34* avec une adresse statique 192.168.110.12 sur VMnet11. Cette configuration est identique sur chaque VM, seule l'adresse *ens34* change.



```

tom@k3s-master:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:25:cf:fd brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.8.168/24 brd 192.168.8.255 scope global dynamic ens33
        valid_lft 1630sec preferred_lft 1630sec
    inet6 fe80::20c:29ff:fe25:cffd/64 scope link
        valid_lft forever preferred_lft forever
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:25:cf:07 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.110.10/24 brd 192.168.110.255 scope global ens34
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe25:cf07/64 scope link
        valid_lft forever preferred_lft forever

```

Fig. 2 – ip a sur k3s-master – ens33 (192.168.8.168) + ens34 (192.168.110.10) – les deux interfaces UP

■ Les deux interfaces sont en état UP avec leurs IPs respectives. L'interface *ens34* porte l'IP 192.168.110.10/24 utilisée par K3s pour toutes les communications inter-nœuds du cluster.


```

tom@k3s-worker-1:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:34:46:10 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.8.169/24 brd 192.168.8.255 scope global dynamic ens33
        valid_lft 1772sec preferred_lft 1772sec
    inet6 fe80::20c:29ff:fe34:4610/64 scope link
        valid_lft forever preferred_lft forever
3: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:34:46:1a brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    inet 192.168.110.11/24 brd 192.168.110.255 scope global ens34
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe34:461a/64 scope link
        valid_lft forever preferred_lft forever

```

Fig. 3 – *ip a* sur *k3s-worker-1* – *ens33* (192.168.8.169) + *ens34* (192.168.110.11)

■ Configuration identique au master : *ens34* porte l'IP 192.168.110.11/24. Le worker peut communiquer avec le master sur 192.168.110.10:6443 pour recevoir ses instructions de l'API Server K3s.

Preuve n°2 – Cluster K3s opérationnel

■ On vérifie que K3s s'installe correctement sur chaque nœud via les logs *systemd*, puis que les 4 nœuds apparaissent en état *Ready* depuis le master.

```

tom@k3s-worker-1:~$ curl -sfl https://get.k3s.io | K3S_URL=https://192.168.110.10:6443 K3S_TOKEN=K10c1c056240a2ceaa2138882a0bf937054d372cd6ea4ef19477d8aa3909746d8e8:
:server:b152b083805f3de9b1e9252a57afac3c sh -
[INFO] Finding release for channel stable
[INFO] Using v1.35.5+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.35.5%2Bk3s1/sha256sum-amd64.txt
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.35.5%2Bk3s1/k3s
[INFO] Verifying binary download
[INFO] Installing k3s to /usr/local/bin/k3s
[INFO] Skipping installation of SELinux RPM
[INFO] Creating /usr/local/bin/kubectrl symlink to k3s
[INFO] Creating /usr/local/bin/crictl symlink to k3s
[INFO] Creating /usr/local/bin/ctr symlink to k3s
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-agent-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s-agent.service.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s-agent.service
[INFO] systemd: Enabling k3s-agent unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s-agent.service -> /etc/systemd/system/k3s-agent.service.
[INFO] Host iptables-save/iptables-restore tools not found
[INFO] Host ip6tables-save/ip6tables-restore tools not found
[INFO] systemd: Starting k3s-agent

```

Fig. 4 – *Installation K3s sur k3s-worker-1* – *v1.35.5+k3s1* téléchargé, *k3s-agent* activé via *systemd*

■ Le script télécharge le binaire *v1.35.5+k3s1*, crée le service *systemd* *k3s-agent* et le démarre. La ligne "systemd: Starting k3s-agent" confirme que le worker contacte le control plane sur 192.168.110.10:6443.

```

tom@k3s-storage:~$ curl -sfl https://get.k3s.io | K3S_URL=https://192.168.110.10:6443 K3S_TOKEN=K10c1c056240a2ceaa2138882a0bf937054d372cd6ea4ef19477d8aa3909746d8e8:
:server:b152b083805f3de9b1e9252a57afac3c sh -
[INFO] Finding release for channel stable
[INFO] Using v1.35.5+k3s1 as release
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.35.5%2Bk3s1/sha256sum-amd64.txt
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.35.5%2Bk3s1/k3s
[INFO] Verifying binary download
[INFO] Installing k3s to /usr/local/bin/k3s
[INFO] Skipping installation of SELinux RPM
[INFO] Creating /usr/local/bin/kubectrl symlink to k3s
[INFO] Creating /usr/local/bin/crictl symlink to k3s
[INFO] Creating /usr/local/bin/ctr symlink to k3s
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh
[INFO] Creating uninstall script /usr/local/bin/k3s-agent-uninstall.sh
[INFO] env: Creating environment file /etc/systemd/system/k3s-agent.service.env
[INFO] systemd: Creating service file /etc/systemd/system/k3s-agent.service
[INFO] systemd: Enabling k3s-agent unit
Created symlink /etc/systemd/system/multi-user.target.wants/k3s-agent.service -> /etc/systemd/system/k3s-agent.service.
[INFO] Host iptables-save/iptables-restore tools not found
[INFO] Host ip6tables-save/ip6tables-restore tools not found
[INFO] systemd: Starting k3s-agent

```

Fig. 5 – *Installation K3s sur k3s-storage* – le nœud rejoint le cluster comme agent de stockage

■ Même procédure sur *k3s-storage*. Ce nœud n'exécutera pas de workloads applicatifs : il est dédié à Longhorn qui y stockera les réplicas des volumes persistants.

Preuve n°3 – Stockage persistant Longhorn

■ On vérifie que Helm déploie Longhorn correctement, que tous les pods sont Running et que le dashboard confirme l'état Healthy des volumes.

```
tom@k3s-master:~$ helm install longhorn longhorn/longhorn --namespace longhorn-system
I0609 19:24:53.082608 25452 warnings.go:107] "Warning: unrecognized format \"int64\""
I0609 19:24:53.082811 25452 warnings.go:107] "Warning: unrecognized format \"int64\""
I0609 19:24:53.083779 25452 warnings.go:107] "Warning: unrecognized format \"int64\""
I0609 19:24:53.084406 25452 warnings.go:107] "Warning: unrecognized format \"int64\""
I0609 19:24:53.097198 25452 warnings.go:107] "Warning: unrecognized format \"int64\""
NAME: longhorn
LAST DEPLOYED: Tue Jun 9 19:24:52 2026
NAMESPACE: longhorn-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Longhorn is now installed on the cluster!

Please wait a few minutes for other Longhorn components such as CSI deployments, Engine Images, and Instance Managers to be initialized.

Visit our documentation at https://longhorn.io/docs/
```

Fig. 6 – helm install longhorn – STATUS: deployed – Longhorn is now installed on the cluster

■ Le statut "STATUS: deployed" confirme que Helm a déployé avec succès l'ensemble des ressources Longhorn. Les warnings "unrecognized format int64" sont inoffensifs.

```
tom@k3s-master:~$ sudo kubectl get pods -n longhorn-system
[sudo] Mot de passe de tom :
NAME                                READY   STATUS    RESTARTS   AGE
csi-attacher-6f9c7bc587-2g2qk       1/1     Running   0           138m
csi-attacher-6f9c7bc587-dnw4v       1/1     Running   0           138m
csi-attacher-6f9c7bc587-vcmkv       1/1     Running   0           138m
csi-provisioner-845c5df44-6nsgd      1/1     Running   0           138m
csi-provisioner-845c5df44-m18p6      1/1     Running   0           138m
csi-provisioner-845c5df44-pvhvk      1/1     Running   0           138m
csi-resizer-6d4df5746-27nbv         1/1     Running   0           138m
csi-resizer-6d4df5746-bndhw         1/1     Running   0           138m
csi-resizer-6d4df5746-x8tsv         1/1     Running   0           138m
csi-snapshotter-55666d7b5-ggkvr     1/1     Running   0           138m
csi-snapshotter-55666d7b5-ggmmx     1/1     Running   0           138m
csi-snapshotter-55666d7b5-kq7qt     1/1     Running   0           138m
engine-image-ei-a4d05f02-5vvsp       1/1     Running   0           139m
engine-image-ei-a4d05f02-7b4db       1/1     Running   0           139m
engine-image-ei-a4d05f02-bg6wt       1/1     Running   0           139m
engine-image-ei-a4d05f02-gd2t2       1/1     Running   0           139m
instance-manager-50d5c167002e7a99d8370330818d701a 1/1     Running   0           139m
instance-manager-8d19e1976b7f7123024deada9e501884 1/1     Running   0           139m
instance-manager-b5c0dae3f03956934482782a8edcd55d 1/1     Running   0           139m
instance-manager-e1919beb4e5587bf7c04b61c1a4c7422 1/1     Running   0           139m
longhorn-csi-plugin-47shx            3/3     Running   0           138m
longhorn-csi-plugin-99t6g            3/3     Running   0           138m
longhorn-csi-plugin-kpvs             3/3     Running   0           138m
longhorn-csi-plugin-vq5fd            3/3     Running   0           138m
longhorn-driver-deployer-cd57959ff-m24cz 1/1     Running   0           140m
longhorn-manager-9ltjs               2/2     Running   0           140m
longhorn-manager-jnzh4               2/2     Running   0           140m
longhorn-manager-mssls               2/2     Running   0           140m
longhorn-manager-tbts4               2/2     Running   0           140m
longhorn-ui-ff5ccfc5f-fxfh9          1/1     Running   0           140m
longhorn-ui-ff5ccfc5f-zbmzv          1/1     Running   0           140m
```

Fig. 7 – kubectl get pods -n longhorn-system – tous les pods Running (CSI, instance-managers, manager)

■ On voit les 4 instance-managers (un par nœud), les 4 longhorn-manager, les drivers CSI. Tous sont en état Running avec 0 restart, ce qui confirme un déploiement stable.

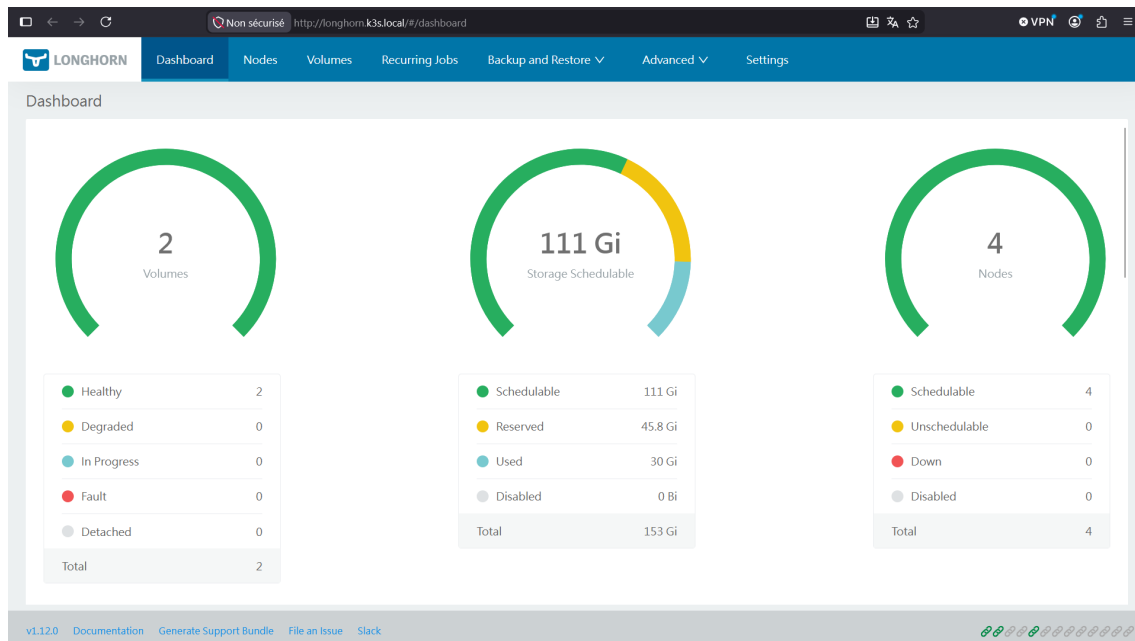


Fig. 8 – Dashboard Longhorn – 2 volumes Healthy, 111 Gi disponibles, 4 nœuds actifs

■ Le dashboard confirme l'état global : 2 volumes Healthy, 111 Gi schedulables sur les 4 nœuds, 30 Gi utilisés. Aucun volume Degraded ni Fault.

Preuve n°4 – Monitoring Grafana + Prometheus

■ On vérifie l'installation du chart, les services dans le namespace monitoring, la configuration des Ingress HTTPS et l'accès à Grafana avec des métriques vivantes.

```
tom@k3s-master:~$ sudo kubectl create namespace monitoring
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack --namespace monitoring
namespace/prometheus-stack created
NAME: kube-prometheus-stack
LAST DEPLOYED: Tue Jun 9 21:48:04 2026
NAMESPACE: monitoring
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
kube-prometheus-stack has been installed. Check its status by running:
  kubectl --namespace monitoring get pods -l "release=kube-prometheus-stack"

Get Grafana 'admin' user password by running:

  kubectl --namespace monitoring get secrets kube-prometheus-stack-grafana -o jsonpath="{.data.admin-password}" | base64 -d ; echo

Access Grafana local instance:

  export POD_NAME=$(kubectl --namespace monitoring get pod -l "app.kubernetes.io/name=grafana,app.kubernetes.io/instance=kube-prometheus-stack" -o name)
  kubectl --namespace monitoring port-forward $POD_NAME 3000

Get your grafana admin user password by running:

  kubectl get secret --namespace monitoring -l app.kubernetes.io/component=admin-secret -o jsonpath="{.items[0].data.admin-password}" | base64 --decode ; echo

Visit https://github.com/prometheus-operator/kube-prometheus for instructions on how to create & configure Alertmanager and Prometheus instances using the Operator.
```

Fig. 9 – helm install kube-prometheus-stack – STATUS: deployed, namespace monitoring créé

■ Le chart installe en une commande Prometheus, Grafana, Alertmanager, node-exporter et kube-state-metrics.

```
tom@k3s-master:~$ sudo kubectl get svc -n monitoring
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
alertmanager-operated	ClusterIP	None	<none>	9093/TCP, 9094/TCP, 9094/UDP	2m
kube-prometheus-stack-alertmanager	ClusterIP	10.43.136.211	<none>	9093/TCP, 8080/TCP	2m6s
kube-prometheus-stack-grafana	ClusterIP	10.43.149.84	<none>	80/TCP	2m6s
kube-prometheus-stack-kube-state-metrics	ClusterIP	10.43.161.184	<none>	8080/TCP	2m6s
kube-prometheus-stack-operator	ClusterIP	10.43.57.205	<none>	443/TCP	2m6s
kube-prometheus-stack-prometheus	ClusterIP	10.43.139.225	<none>	9090/TCP, 8080/TCP	2m6s
kube-prometheus-stack-prometheus-node-exporter	ClusterIP	10.43.188.103	<none>	9100/TCP	2m6s
prometheus-operated	ClusterIP	None	<none>	9090/TCP	119s

Fig. 10 – kubectl get svc -n monitoring – ClusterIPs Grafana (80/TCP), Prometheus (9090/TCP), Alertmanager (9093/TCP)

■ Les services sont en ClusterIP, accessibles uniquement depuis l'intérieur du cluster. C'est pour cette raison qu'on crée des Ingress avec Traefik pour les exposer en HTTPS vers l'extérieur.

```
GNU nano 7.2 grafana-ingress.yaml *
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: grafana-ingress
  namespace: monitoring
  annotations:
    traefik.ingress.kubernetes.io/router.entrypoints: websecure
    cert-manager.io/cluster-issuer: selfsigned-issuer
spec:
  tls:
  - hosts:
    - grafana.k3s.local
    secretName: grafana-tls
  rules:
  - host: grafana.k3s.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: kube-prometheus-stack-grafana
            port:
              number: 80
```

Fig. 11 – grafana-ingress.yaml – annotations Traefik websecure + Cert-Manager selfsigned-issuer

■ L'annotation traefik.ingress.kubernetes.io/router.entrypoints: websecure indique à Traefik d'exposer le service sur HTTPS (port 443). L'annotation cert-manager.io/cluster-issuer: selfsigned-issuer déclenche la génération automatique du certificat TLS.

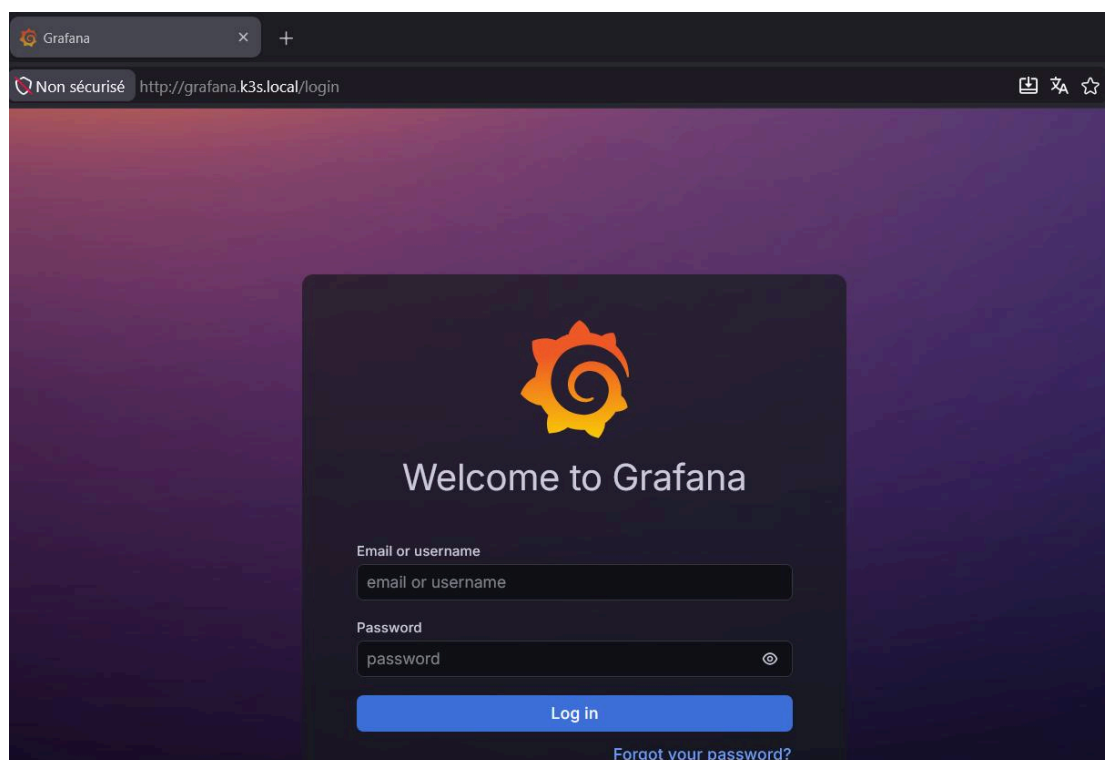


Fig. 12 – Page de login Grafana – accessible via <https://grafana.k3s.local> – certificat TLS actif

■ Grafana est accessible depuis Windows via grafana.k3s.local (entrée ajoutée dans le fichier hosts). L'alerte "Non sécurisé" est attendue pour un certificat auto-signé – en production on utiliserait Let's

Encrypt.

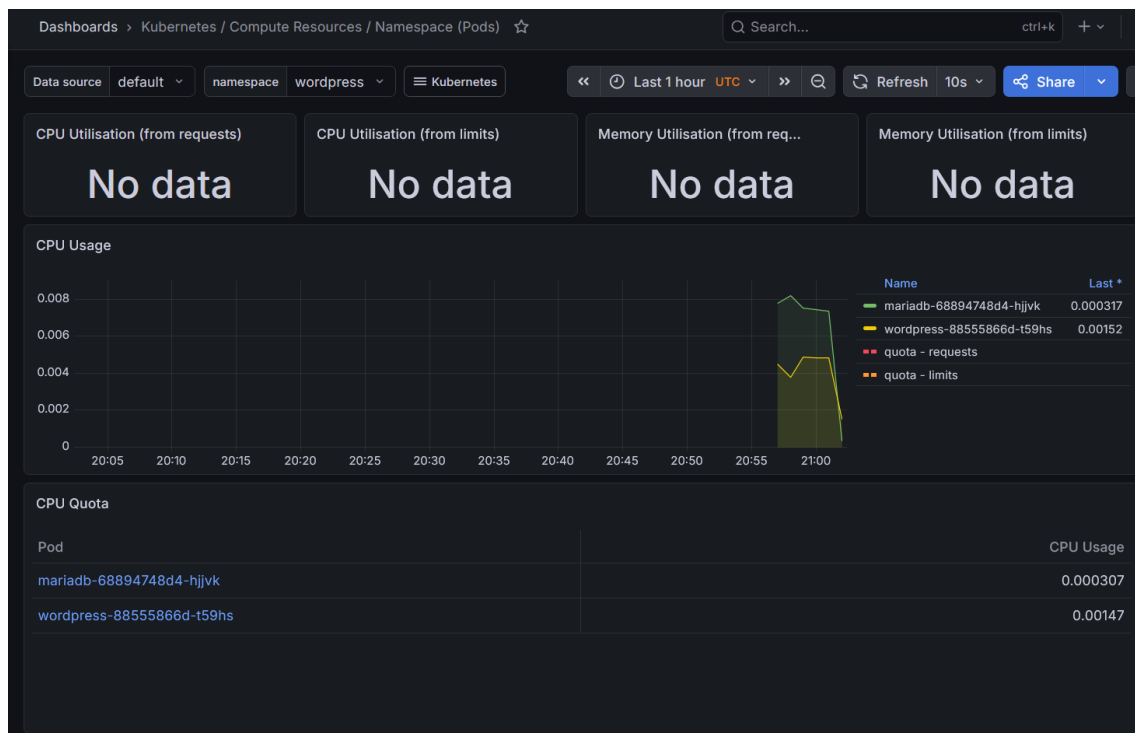


Fig. 13 – Dashboard Grafana – namespace wordpress – métriques CPU pods WordPress et MariaDB en temps réel

■ Le dashboard Kubernetes / Compute Resources / Namespace montre la consommation CPU des pods wordpress et mariadb. Les courbes confirment que Prometheus collecte bien leurs métriques via node-exporter.

Preuve n°5 – Pipeline GitOps – Gitea + ArgoCD

■ On vérifie qu'ArgoCD détecte le dépôt Gitea et synchronise automatiquement les applications. Le statut "Healthy + Synced" confirme que l'état du cluster correspond aux manifests Git.

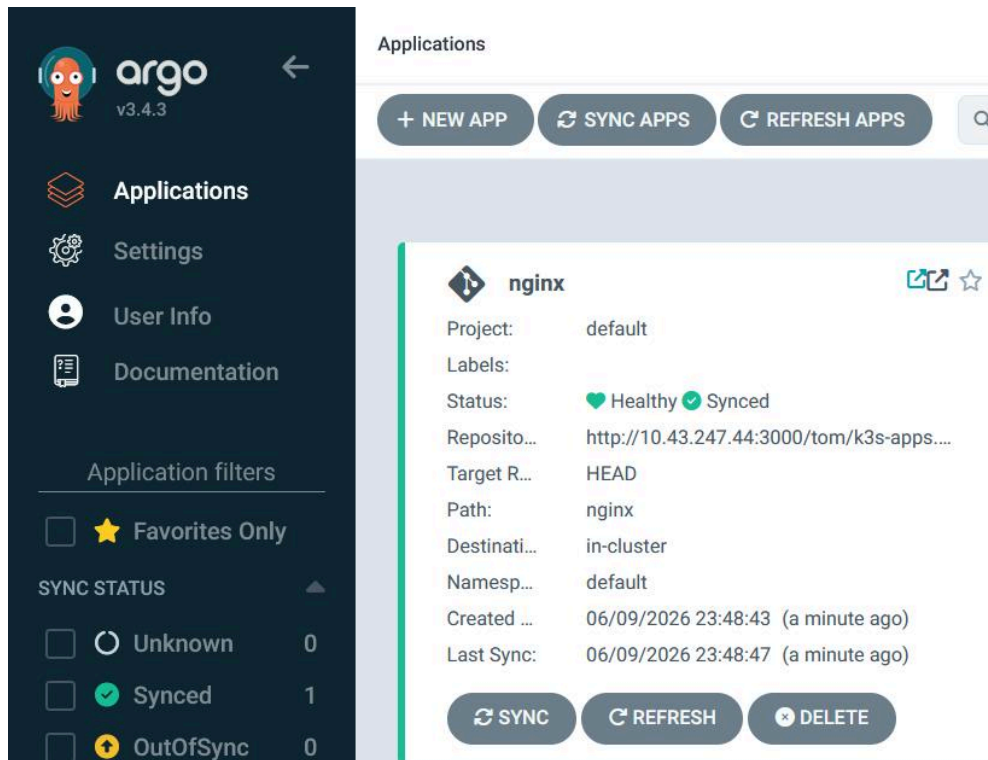


Fig. 14 – ArgoCD – application nginx • Healthy • Synced – synchronisation automatique active

■ "Healthy" signifie que tous les pods sont Running. "Synced" signifie que l'état du cluster correspond exactement aux manifests dans Gitea. "Last Sync" montre que la synchronisation automatique fonctionne.

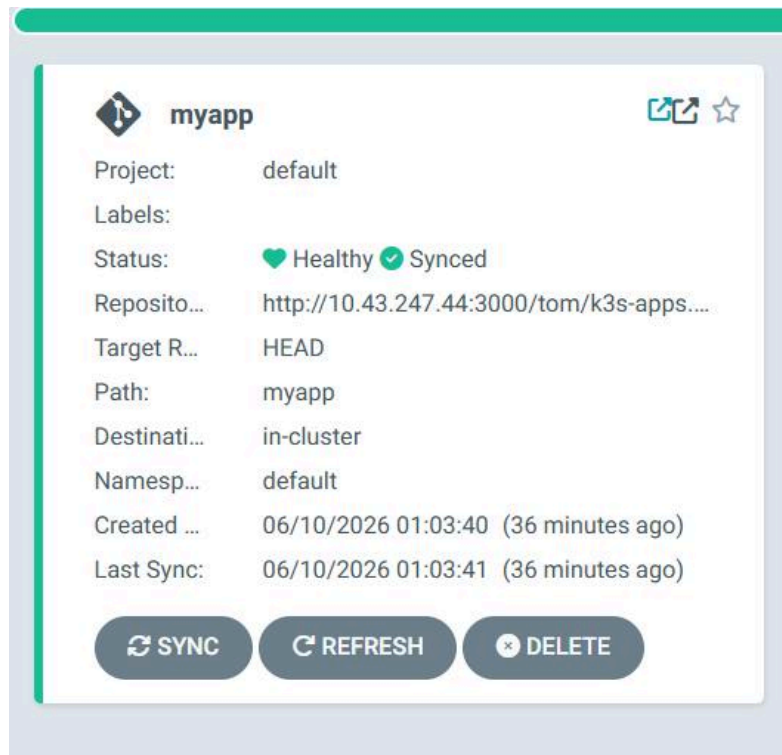


Fig. 15 – ArgoCD détail myapp – Healthy + Synced to HEAD – dépôt k3s-apps sur Gitea

■ La carte montre le lien ArgoCD ↔ Gitea : dépôt source <http://10.43.247.44:3000/tom/k3s-apps>, chemin /myapp, référence HEAD. À chaque commit, ArgoCD synchronise automatiquement le cluster.

Preuve n°6 – Registry privée Harbor + scan CVE Trivy

■ On vérifie le cycle DevSecOps complet : Harbor déployé, image v1 poussée et CVE détectée, image v2 corrigée via apk upgrade avec re-scan propre.

```
tom@k3s-master:~$ helm install harbor harbor/harbor \
--namespace harbor \
--set expose.type=ingress \
--set expose.ingress.hosts.core=harbor.k3s.local \
--set expose.tls.enabled=true \
--set expose.tls.certSource=secret \
--set externalURL=https://harbor.k3s.local \
--set persistence.persistentVolumeClaim.registry.storageClass=longhorn \
--set persistence.persistentVolumeClaim.jobservice.storageClass=longhorn \
--set persistence.persistentVolumeClaim.database.storageClass=longhorn \
--set persistence.persistentVolumeClaim.redis.storageClass=longhorn \
--set persistence.persistentVolumeClaim.trivy.storageClass=longhorn
NAME: harbor
LAST DEPLOYED: Wed Jun 10 00:16:28 2026
NAMESPACE: harbor
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
Please wait for several minutes for Harbor deployment to complete.
Then you should be able to visit the Harbor portal at https://harbor.k3s.local
For more details, please visit https://github.com/qoharbor/harbor
```

Fig. 16 – helm install harbor – STATUS: deployed – PVCs Longhorn créés pour la registry

■ Harbor est installé avec Longhorn comme StorageClass pour tous ses PVCs (registry, jobservice, database, redis, trivy). Cela garantit la persistance des images et des résultats de scan.

```
tom@k3s-master:~/myapp$ docker build -t harbor.k3s.local/k3s-lab/myapp:v1 .
docker push harbor.k3s.local/k3s-lab/myapp:v1
Sending build context to Docker daemon 3.072kB
Step 1/2 : FROM nginx:alpine
alpine: Pulling from library/nginx
6a0ac1617861: Pull complete
abaae85d1626: Pull complete
43f834d60d8a: Pull complete
de1b677d8c00: Pull complete
94d083cf706a: Pull complete
e654dbbbb9e1: Pull complete
a5008f4a4b25: Pull complete
fbaed3f7fcbe: Pull complete
Digest: sha256:8b1e78743a03dbb2c95171cc58639fef29abc8816598e27fb910ed2e621e589a
Status: Downloaded newer image for nginx:alpine
--> da954fb959a3
Step 2/2 : COPY index.html /usr/share/nginx/html/index.html
--> 113fd414039a
Successfully built 113fd414039a
Successfully tagged harbor.k3s.local/k3s-lab/myapp:v1
The push refers to repository [harbor.k3s.local/k3s-lab/myapp]
7122f63ac2f0: Pushed
5519239b187c: Pushed
9a2cbc774b1f: Pushed
ad8253b95bde: Pushed
906af8ee0f8a: Pushed
2d9b3b001567: Pushed
2b05bd5bca95: Pushed
b910427bd94b: Pushed
29df493baa13: Pushed
v1: digest: sha256:a1be191269716a8e7289c7069809ca69de2fe1def994b175b912f8c2d8f09b4d size: 2196
```

Fig. 17 – docker build + push harbor.k3s.local/k3s-lab/myapp:v1 – tous les layers poussés, digest SHA256 confirmé

■ L'image est construite depuis une base nginx:alpine. Dès la fin du push, Trivy lance automatiquement un scan CVE – la CVE-2026-6732 (Haute) sera détectée dans les paquets Alpine non mis à jour.

```
tom@k3s-master:~/myapp$ docker build -t harbor.k3s.local/k3s-lab/myapp:v2 .
docker push harbor.k3s.local/k3s-lab/myapp:v2
Sending build context to Docker daemon 3.072kB
Step 1/3 : FROM nginx:alpine
--> da954fb959a3
Step 2/3 : RUN apk upgrade --no-cache
--> Running in 6f39d4e2605a
(1/2) Upgrading aom-libs (3.13.1-r1 -> 3.14.1-r0)
(2/2) Upgrading libxml2 (2.13.9-r0 -> 2.13.9-r1)
OK: 59.0 MiB in 71 packages
Removing intermediate container 6f39d4e2605a
--> f09d72b8b36b
Step 3/3 : COPY index.html /usr/share/nginx/html/index.html
--> 786cf91d481f
Successfully built 786cf91d481f
Successfully tagged harbor.k3s.local/k3s-lab/myapp:v2
The push refers to repository [harbor.k3s.local/k3s-lab/myapp]
f3a3d3fd3401: Pushed
bb92aa648419: Pushed
5519239b187c: Layer already exists
9a2cbc774b1f: Layer already exists
ad8253b95bde: Layer already exists
906af8ee0f8a: Layer already exists
2d9b3b001567: Layer already exists
2b05bd5bca95: Layer already exists
b910427bd94b: Layer already exists
29df493baa13: Layer already exists
v2: digest: sha256:afafc424a63bc9de751cf4a33e40a2e69cdf99b600d3dc69ae41a428a3cd1168 size: 2407
```

Fig. 18 – docker build myapp:v2 – RUN apk upgrade --no-cache corrige CVE-2026-6732 – push réussi

■ L'ajout de RUN apk upgrade --no-cache met à jour tous les paquets Alpine. On voit la mise à jour des librairies vulnérables (aom-libs, libxml2). Le re-scan Trivy sur v2 ne détecte aucune vulnérabilité : le cycle DevSecOps est complet.

Preuve n°7 – Politiques de sécurité Kyverno

■ On vérifie que Kyverno est installé et que la ClusterPolicy est active en mode Enforce. La tentative de créer un pod root doit être bloquée immédiatement par l'Admission Webhook.

```
tom@k3s-master:~$ sudo chmod 644 /etc/rancher/k3s/k3s.yaml
helm install kyverno kyverno/kyverno --namespace kyverno --create-namespace
NAME: kyverno
LAST DEPLOYED: Wed Jun 10 23:10:47 2026
NAMESPACE: kyverno
STATUS: deployed
REVISION: 1
NOTES:
Chart version: 3.8.1
Kyverno version: v1.18.1

Thank you for installing kyverno! Your release is named kyverno.

The following components have been installed in your cluster:
- CRDs
- Admission controller
- Reports controller
- Cleanup controller
- Background controller

⚠ WARNING: Setting the admission controller replica count below 2 means Kyverno is not running in high availability mode.

⚠ WARNING: PolicyExceptions are disabled by default. To enable them, set '--enablePolicyException' to true.

💡 Note: There is a trade-off when deciding which approach to take regarding Namespace exclusions. Please see the documentation at https://kyverno.io/docs/installation/#security-vs-operability to understand the risks.
```

Fig. 19 – helm install kyverno v1.18.1 – Admission Controller, CRDs et Reports controller actifs

■ Kyverno installe 5 composants : CRDs, Admission Controller (intercepte les requêtes), Reports Controller, Cleanup Controller et Background Controller. Le WARNING sur le replica count est attendu en lab mono-réplica.


```

GNU nano 7.2 /home/tom/kyverno-policy.yaml *
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: disallow-root-user
spec:
  validationFailureAction: Enforce
  rules:
  - name: check-runAsNonRoot
    match:
      resources:
        kinds:
        - Pod
    validate:
      message: "Les conteneurs ne peuvent pas tourner en root."
      pattern:
        spec:
          containers:
          - securityContext:
              runAsNonRoot: true

```

Fig. 20 – kyverno-policy.yaml – ClusterPolicy disallow-root-user – validationFailureAction: Enforce

■ La politique cible tous les pods et vérifie que securityContext.runAsNonRoot est à true. En mode Enforce, toute tentative de créer un pod sans cette contrainte est rejetée immédiatement.

Preuve n°8 – Load Balancer bare-metal MetalLB

■ On vérifie que MetalLB assigne bien une IP de la plage 192.168.110.100-120 à un service LoadBalancer, et que cette IP est directement accessible depuis Windows sans aucune règle NAT.

```

tom@k3s-master:~$ sudo kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.14.5/config/manifests/metallb-native.yaml
namespace/metallb-system created
customresourcedefinition.apiextensions.k8s.io/bfdprofiles.metallb.io created
customresourcedefinition.apiextensions.k8s.io/bgpadvertisements.metallb.io created
customresourcedefinition.apiextensions.k8s.io/bgppeers.metallb.io created
customresourcedefinition.apiextensions.k8s.io/communities.metallb.io created
customresourcedefinition.apiextensions.k8s.io/ipaddresspools.metallb.io created
customresourcedefinition.apiextensions.k8s.io/l2advertisements.metallb.io created
customresourcedefinition.apiextensions.k8s.io/serviceL2statuses.metallb.io created
serviceaccount/controller created
serviceaccount/speaker created
role.rbac.authorization.k8s.io/controller created
role.rbac.authorization.k8s.io/pod-lister created
clusterrole.rbac.authorization.k8s.io/metallb-system:controller created
clusterrole.rbac.authorization.k8s.io/metallb-system:speaker created
rolebinding.rbac.authorization.k8s.io/controller created
rolebinding.rbac.authorization.k8s.io/pod-lister created
clusterrolebinding.rbac.authorization.k8s.io/metallb-system:controller created
clusterrolebinding.rbac.authorization.k8s.io/metallb-system:speaker created
configmap/metallb-excludeI2 created
secret/metallb-webhook-cert created
service/metallb-webhook-service created
deployment.apps/controller created
daemonset.apps/speaker created
validatingwebhookconfiguration.admissionregistration.k8s.io/metallb-webhook-configuration created

```

Fig. 21 – kubectl apply metallb-native.yaml – CRDs IPAddressPool + L2Advertisement créés

■ MetalLB crée ses CRDs (IPAddressPool, L2Advertisement, BGPPeer) et déploie le contrôleur (assigne les IPs) et le speaker (répond aux requêtes ARP en mode Layer 2).

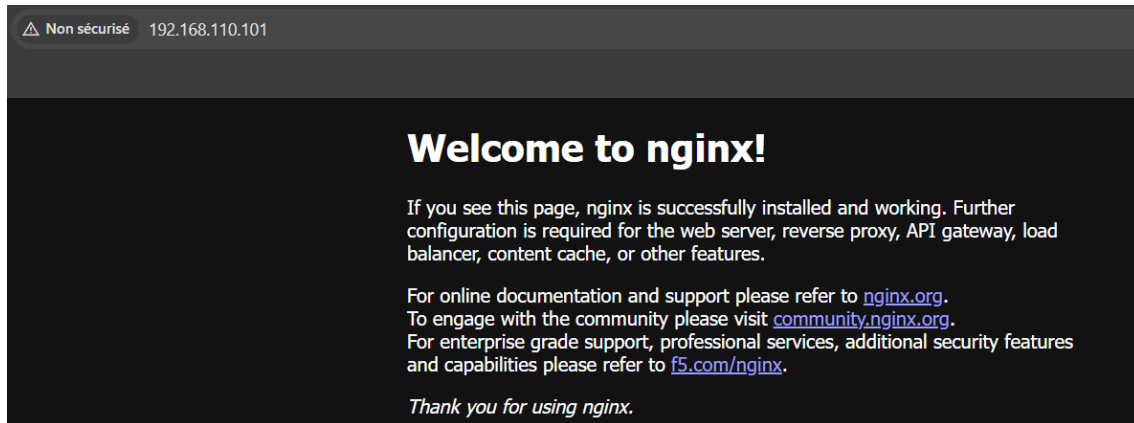


Fig. 22 – <http://192.168.110.101> – Welcome to nginx! – service accessible via IP externe MetalLB

■ La page nginx est accessible directement depuis Windows via 192.168.110.101, sans NAT ni port-forwarding. MetalLB a assigné cette IP du pool 192.168.110.100-120 et le speaker l'annonce via ARP sur VMnet11.

6. Bilan et Compétences

Difficultés rencontrées

Difficulté	Solution
Helm ne trouvait pas le kubeconfig	export KUBECONFIG=/etc/rancher/k3s/k3s.yaml + chmod 644
Workers ne pouvaient pas puller Harbor	registries.yaml sur chaque worker + projet Harbor en public
Conflit volumes Longhorn (FailedAttachVolume)	Suppression complète et recréation propre des ressources
RAM saturée (Grafana + Prometheus)	Limits : Grafana 256 Mi, Prometheus 300 Mi
ArgoCD inaccessible via Ingress HTTPS	Patch --insecure + Traefik HTTPS frontal
IPs DHCP multiples sur ens33	sudo reboot pour nettoyer les baux DHCP

Compétences BTS SIO SISR validées

Bloc	Compétence	Mise en œuvre
B1.1	Gérer le patrimoine	Installation 4 VMs Debian 12, services systemd, réseau statique dual-interface
B1.2	Répondre aux incidents	Diagnostic et résolution : Longhorn FailedAttachVolume, Helm kubeconfig, ArgoCD HTTPS
B1.3	Présence en ligne	WordPress + MariaDB, PVC Longhorn, Ingress HTTPS
B1.4	Mode projet	Gestion des dépendances, déploiement ordonné des 14 composants
B1.5	Services	K3s, Longhorn, Grafana, ArgoCD, Harbor, WordPress – tous en HTTPS
Cyber	Sécurité (CIA)	Confidentialité : Kyverno + Harbor. Intégrité : Trivy CVE. Disponibilité : Longhorn

Compétences acquises

- Maîtrise de l'administration d'un cluster Kubernetes multi-nœuds (K3s)
- Déploiement d'applications complexes via Helm (14 composants)
- Mise en pratique du GitOps : Gitea + ArgoCD (synchronisation automatique)
- Application des pratiques DevSecOps : scan CVE Trivy, politiques admission Kyverno
- Gestion du réseau virtualisé : dual-interface, MetalLB, Ingress Traefik
- Diagnostic et résolution de problèmes en environnement K8s (6 incidents résolus)